

Administración de usuarios y permisos especiales

La base de datos interna de los usuarios en GNU/Linux está relacionada con dos archivos:

- ~ /etc/passwd contiene los nombres de los usuarios
- ~ /etc/shadow contiene la contraseña encriptada con SHA512

El proceso de login de usuarios y servicios está relacionado con PAM (Pluggable Authentication Module). Cada aplicación o servicio con capacidades PAM tiene un archivo en el directorio /etc/pam.d/.

~ Cada archivo en este directorio tiene el mismo nombre del servicio al que controla el acceso.

~ Si una aplicación utiliza PAM ella es la responsable por definir su nombre de servicio e instalar su propio archivo de configuración en el directorio /etc/pam.d/.

1. Identidad y Archivos Críticos (Anatomía del Acceso)

No basta con saber qué hacen los archivos, hay que saber leerlos como un auditor:

~ /etc/passwd: Es público (lectura para todos). Cada línea tiene 7 campos separados por dos puntos (:):

User1:x:1001:1001:User1 Admin:/home/User1:/bin/bash

1. ~ User1: Nombre de login
2. ~ x: indica que la clave está protegida en /etc/shadow. (si hubiera un hash aca, seria una vulnerabilidad grave)
3. ~ 1001: UID (el User ID). El 0 es exclusivo de root. 1 a 999 son servicios.
4. ~ 1001: GID (Group ID) principal.
5. ~ User1 Admin: GECOS (comentarios, nombre real, teléfono).
6. ~ /home/User1: Directorio de trabajo.
7. ~ /bin/bash: La shell por defecto. (Si un usuario tiene /bin/false o /sbin/nologin, significa que la cuenta existe pero no puede iniciar sesión interactiva, ideal para cuentas de servicios).

2. Gestión de Cuentas Avanzada

Comandos con buenas prácticas para cuando administras servidores. Aquí hay algunos modificadores importantes:

Creación:

```
useradd -m -s /bin/bash -c "Auditor Externo" auditor1
```

(Crea el usuario, le fuerza la creación del directorio home -m, le asigna bash como shell -s y le pone un comentario -c).

Modificación: Para agregarlo a grupos

```
~usermod -a -G grupo usuario
```

Bloquear una cuenta si detectas una intrusión:

```
~usermod -L usuario (Lock: bloquea la contraseña)
```

```
~usermod -U usuario (Unlock: la desbloquea).
```

Auditoría de contraseñas (chage):

Visualizar el estado: `chage -l usuario`

Comando de Hardening:

```
~chage -M 90 -W 7 -I 5 usuario
```

(Forzar cambio cada 90 días, avisar 7 días antes, bloquear la cuenta si pasan 5 días de gracia)

3. Matemáticas de Permisos (rwx) y Umask

En entornos Linux, los permisos se leen en Octal (números).

La base matemática es: Lectura (r=4), Escritura (w=2), Ejecución (x=1).

Por ejemplo: `chmod 750 archivo` = Dueño tiene todo (7), Grupo lee y ejecuta (5), Otros no tienen acceso (0).

La primera terna de caracteres (rwx) indica que el usuario dueño posee permisos de lectura (r), escritura (w), y ejecución (x) sobre el directorio. La segunda terna (r-x) indica que los miembros del grupo propietario poseen permisos de lectura y ejecución (pero no de escritura). Finalmente, la última terna muestra que todos los demás usuarios del sistema no poseen ningún privilegio.

Al crear un archivo o un directorio, se establecen por defecto los permisos `rw-rw-r-` (octal 664) y `rw-rw-r-x` (octal 775), respectivamente. Si lo hacemos como root, los permisos se setean como `rw-r-r-` (octal 644) y `rw-r-xr-x` (octal 755). Esto es resultado de aplicar lo que se conoce como máscaras de archivos y directorios, cuyos valores podemos visualizar y cambiar mediante el comando `umask`.

Ejecución en Directorios vs Archivos:

En un archivo, x significa que puedes correrlo como un programa o script.

En un directorio, x significa que puedes atravesarlo (hacer `cd` hacia él). Si tienes permiso de lectura r en una carpeta pero no de ejecución x, puedes ver qué archivos hay (con `ls`), pero no puedes leer su contenido.

Umask (El filtro invisible): Si la máscara es 0022 y creas un archivo (máximo 666): $666 - 022 = 644$ (`rw- r-- r--`). Así nacen los archivos seguros

PERMISOS Y ATRIBUTOS ESPECIALES (SUID y SGID)

Este es el territorio donde ocurren la mayoría de los hackeos a nivel local.

~ SUID (Set Owner User ID - Valor octal 4000): Si un archivo tiene SUID, al ejecutarlo, obtienes temporalmente los permisos de su creador.

Aplicación: `chmod u+s /usr/bin/comando` o `chmod 4755 comando`.

Vista: En un `ls -l`, la "x" del dueño se vuelve una "s" minúscula (ej: -rwsr-xr-x).

~ SGID (Set Group ID - Valor octal 2000): En archivos funciona igual que el SUID pero a nivel de grupo. Pero en directorios tiene un uso brillante: si le aplicas SGID a una carpeta compartida (`chmod g+s carpeta`), cualquier archivo que se cree adentro heredará el grupo de la carpeta, no el grupo del usuario que lo creó. Ideal para carpetas de trabajo en equipo.

Si un programa posee el permiso SUID y root es el propietario de dicho archivo, el proceso que resulte de ejecutarlo tendrá permisos de superusuario y contará con los mismos accesos de root en todo el sistema.

Esto pone de relieve la necesidad de restringir al mínimo posible los archivos que cuenten con este permiso especial. Generalmente, el único programa que cuenta con SUID es `passwd`, el cual permite que cada usuario del sistema pueda cambiar su propia contraseña (por defecto, no la de los demás) sin tener que depender para ello de un administrador.

Para identificar todos los archivos que cuenten con los permisos de SUID o SGID:

```
find /usr/bin -type f -perm -u+s
```

```
find /usr/bin -type f -perm -g+s
```

Podríamos guardar la salida de los dos comandos anteriores en archivos de texto y luego de un tiempo volver a ejecutarlos. De esta manera, podríamos compararlos y asegurarnos de que los permisos especiales no hayan sido establecidos en ningún otro programa desde la última revisión.

ACLs (Access Control Lists)

Los archivos y directorios tienen conjuntos de permisos configurados para el propietario del archivo, el grupo propietario del archivo y todos los otros usuarios del sistema. Sin embargo, estos permisos tienen sus limitaciones. Por ejemplo, no podemos configurar diferentes permisos para usuarios diferentes sobre un mismo archivo o directorio.

Las ACLs rompen la regla de "Dueño/Grupo/Otros" permitiéndote dar acceso a Juan y a María, con permisos distintos, sin meterlos en un grupo común.

~ Ver reglas: `getfacl archivo.txt`

~ Dar permiso específico: `setfacl -m u:juan:rw archivo.txt`
(Le da lectura y escritura solo a Juan).

~ Quitar permiso específico: `setfacl -x u:juan archivo.txt`

~ Borrar todas las reglas ACL: `setfacl -b archivo.txt`

Nota visual: Si un archivo tiene ACLs activas, al hacer `ls -l` verás un signo más (+) al final de los permisos convencionales.
(ej: -rw-r--r--+).

Atributos Extendidos (Buenas practicas)

Controlados por el sistema de archivos (Ext4, XFS), actúan por debajo del usuario root.

~ Inmutable (+i): `chattr +i archivo.conf`. Bloquea el archivo por completo. Protege configuraciones críticas de red o de DNS (ej. /etc/resolv.conf) para que ningún script malicioso o error humano lo altere.

~ Append Only (+a): `chattr +a /var/log/auth.log`. Fundamental para la evidencia forense. El archivo crece, pero su historia no se puede reescribir ni borrar.

Perspectiva de Ciberseguridad

En una evaluación de seguridad (Pentesting) interna, o durante una auditoría defensiva (Blue Team):

1. Búsqueda de escalada de privilegios: Lo primero que hace un atacante al comprometer a un usuario normal es buscar archivos mal configurados ejecutando por ejemplo: `find / -perm -4000 2>/dev/null`. Si encuentra un binario como nmap, vim o python con el bit SUID activado y propiedad de root, puede usarlo para ejecutar un comando y convertirse en superusuario instantáneamente (Técnica documentada en GTFOBins).
2. Archivos huérfanos: Cuando usas `userdel` sin el parámetro `-r`, quedan archivos en el sistema cuyo dueño ahora es un número UID que no existe (ej: "1005"). Esto es un riesgo de seguridad (archivos huérfanos) que debe ser auditado.
3. Defensa de Logs: El uso de `chattr +a` es un seguro de vida contra el Ransomware y los intrusos que intentan cubrir sus huellas borrando los archivos de `/var/log`.

Por último, un resumen rápido para tener a mano.

